DEPARTAMENTO:	**CIENCIAS DE LA COMPUTACIÓN**	**CARRERA:**	**INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN EN LÍNEA**		
ASIGNATURA:	Aplicaciones Distribuidas	**PERÍODO LECTIVO:**	202651	**NIVEL:**	7mo
DOCENTE:	Ing. Luis Castillo, Mgtr.	**NRC:**	37869	**PRÁCTICA N°:**	1
LABORATORIO DONDE SE DESARROLLARÁ LA PRÁCTICA		Virtual			
TEMA DE LA PRÁCTICA:	Sockets (TCP/UDP)				

INTRODUCCIÓN:

En esta actividad, se desarrollará un ejemplo práctico del manejo de sockets, tanto TCP como UDP. Se creará un servidor, tanto TCP como UDP para la creación de hilos de ejecución por cada cliente que se conecte. Por otro lado se creará un cliente, tanto TCP como UDP para la comunicación cliente-servidor y el intercambio de mensajes.

OBJETIVOS:

- Implementar un servidor TCP y UDP para el uso de hilos y manejo de múltiples usuarios.
- Implementar un cliente TCP y UDP para la conexión al servidor respectivo (múltiples usuarios).
- Manejar una comunicación bidireccional entre cliente servidor.

MATERIALES:

REACTIVOS: No aplica	**INSUMOS:** ● Una PC con Windows/Linux ● Acceso a Internet ● Editor de texto ● NodeJS

EQUIPOS:

Windows 10 o superior, Procesador Intel® Core™ i7-6700T o superior, 12GB RAM o superior, 480GB SSD o superior, Intel HD Graphics 530, similar o superior.

MUESTRA:
No aplica

INSTRUCCIONES:

1. Utilizar como material principal de apoyo, aquel indicado en clase por el docente
2. Revisa atentamente lo solicitado en el apartado de actividades a desarrollar
3. No olvide incluir capturas de pantallas de todas las actividades realizadas durante la práctica.

ACTIVIDADES POR DESARROLLAR:

PARTE 1: Implementación del servidor TCP

Paso 1: Crear un archivo que funcione como servidor TCP.

a) Crear un archivo que contenga al servidor, ejemplo “tcp_server.js”
b) Usar el módulo 'net' de Node.js para crear el servidor TCP
c) Crear un Evento que se dispare cuando el cliente envía datos
d) Manejar una condición, si el mensaje es 'salir', cierra la conexión con el cliente
e) Manejar una comunicación bidireccional donde el servidor responda el mismo mensaje en mayúsculas

Paso 2: Ejecutar el servidor TCP.

Ejecutar el servidor usando el comando *node NombreArchivoServidor*

PARTE 2: Implementación del cliente TCP

Paso 1: Crear un archivo que funcione como cliente TCP.

a) Crear un archivo que ejecute a los clientes, ejemplo “tcp_cliente.js”
b) Importar los módulos necesarios: 'net' para la conexión TCP y 'readline' para la entrada por consola
c) Crea un nuevo socket TCP (cliente)
d) Configurar readline para leer líneas desde la consola
e) Conecta al servidor en *localhost:#puerto (12345)*
f) Capturar cada línea escrita por el usuario
g) Enviar la línea al servidor
h) Controlar, si escribe 'salir', cierra readline
i) Cuando recibe datos del servidor, los muestra por consola
j) Manejar un mensaje cuando la conexión se cierra

Paso 2: Ejecutar los clientes TCP.

Ejecutar los usuarios en varias terminales usando el comando *node NombreArchivoCliente*

Paso 3: Revisar la comunicación cliente-servidor

Conectar un cliente con el servidor y revisar la comunicación entre ambos.
Conectar dos o más clientes con el servidor y revisar la comunicación entre los diferentes actores

PARTE 3: Implementación del servidor UDP

Paso 1: Crear un archivo que funcione como servidor UDP.

a) Crear un archivo que contenga al servidor, ejemplo “udp_server.js”
b) Usar el módulo 'dgram' de Node.js para crear sockets UDP
c) Crear un socket UDP de tipo IPv4
d) Cada vez que se recibe un mensaje el mismo se debe imprimir en consola junto con la IP y el puerto
e) Se debe controla que, si el mensaje es 'salir', no responda nada
f) Se contará el número de palabras (separadas por espacios)
g) Enviar la respuesta al cliente (a su dirección y puerto)
h) El servidor debe quedar a la escucha en el puerto especificado

Paso 2: Ejecutar el servidor UDP.

Ejecutar el servidor usando el comando *node NombreArchivoServidor*

PARTE 4: Implementación del cliente UDP

Paso 1: Crear un archivo que funcione como cliente UDP.

a) Crear un archivo que ejecute a los clientes, ejemplo “udp_cliente.js”
b) Importar los módulos necesarios: 'dgram' para UDP, 'readline' para entrada por consola
c) Crear el socket UDP del cliente
d) Configurar readline para leer líneas por consola
e) Definir los datos del servidor al que se enviarán los mensajes
f) Enviar el mensaje al servidor
g) Si escribe 'salir', cerrar la interfaz y el socket
h) Mostrar la respuesta recibida del servidor

Paso 2: Ejecutar los clientes UDP.

Ejecutar los usuarios en varias terminales usando el comando *node NombreArchivoCliente*

Paso 3: Revisar la comunicación cliente-servidor

Conectar un cliente con el servidor y revisar la comunicación entre ambos.
Conectar dos o más clientes con el servidor y revisar la comunicación entre los diferentes actores

SECCIÓN DE PREGUNTAS/ACTIVIDADES

1. Chat Múltiple con Identificación de Usuario (TCP)
    a. Transformar el sistema cliente-servidor TCP en una versión básica de **chat grupal**, donde múltiples clientes puedan enviar mensajes al servidor y este los reenvíe a todos los demás.
        i. Al conectarse, cada cliente debe **enviar un nombre de usuario** como primer mensaje.
        ii. El servidor debe **asociar el socket con ese nombre**.
    b. Consideraciones
        i. En el servidor, usar un Map<socket, nombre> para llevar seguimiento.
        ii. Implementar una función broadcast (mensaje, emisor) para enviar a todos menos al emisor.
        iii. Agregar un prefijo en el cliente como [YO] para distinguir los propios mensajes.
        iv. Probar con tres terminales (servidor y dos clientes)

RESULTADOS OBTENIDOS:

a. Realizar el informe en el formato general de informes de laboratorio.
b. Evidencia con capturas las actividades prácticas realizadas.

c. Anexar el código fuente en el informe.

CONCLUSIONES:

- Escribir al menos dos conclusiones.

RECOMENDACIONES:

- Escribir al menos dos recomendaciones.

FIRMAS

F: ..... **Nombre: Ing. Luis Castillo, Mgtr.** **DOCENTE**	**F: .....** **Nombre: Ing. Juan Fernando Galarraga, Mgtr.** **COORDINADOR DE ÁREA DE CONOCIMIENTO**	**F: .....** **Nombre: Crnl (SP) Fidel Castro de la Cruz** **JEFE DE LABORATORIO**